Spam Email Detection Using Statistical Inference and Machine Learning

Student: Om Shinde
Topic: Spam Email Detection
Dataset: CSV file uploaded in Google Colab
Target: Spam / Ham (not spam)

Project aim

Analyze email/message text, explore spam patterns, apply statistical inference practicals, compare classification models, and build a small prediction interface.

Important notes

- This notebook is designed for **Google Colab**.
- Run the cells from top to bottom and upload a CSV dataset containing email/message text and a spam label.
- It supports common column names such as `v1`/`v2`, `Category`/`Message`, `label`/`text`, and `spam`/`email`.
- The label should identify messages as spam or ham/not spam. The notebook normalizes common label variations.
- Statistical tests show associations in the dataset; they do not prove that a word causes a message to be spam.

1. Import libraries and upload the dataset

What we use

- `pandas`: read and manage the dataset.
- `numpy`: numerical calculations.
- `matplotlib` and `seaborn`: graphs.
- `scipy.stats`: statistical tests.
- `scikit-learn`: text processing and machine learning.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings

from scipy import stats
from google.colab import files
from IPython.display import display

sns.set_theme(style="whitegrid")
warnings.filterwarnings("ignore")

uploaded = files.upload()
filename = next(iter(uploaded))
df = pd.read_csv(filename, encoding="utf-8", encoding_errors="replace")

print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nFirst five rows:")
display(df.head())
print("\nData types:")
print(df.dtypes)
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving emails.csv to emails (1).csv
Dataset shape: (5728, 2)

Column names:
['text', 'spam']

First five rows:
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

```
Data types:
text    object
spam     int64
```

2. Check data quality and clean the dataset

What we use

- `isnull().sum()`: counts missing values.
- `duplicated().sum()`: counts duplicate rows.
- Column-name detection: identifies the message and label columns.
- Label normalization: converts common labels into `spam` and `ham`.

```
print("Missing values before cleaning:")
display(df.isnull().sum().to_frame("Missing count"))
print("Duplicate rows before cleaning:", df.duplicated().sum())

df = df.drop_duplicates().copy()
# Detect likely message and label columns from common public spam datasets.
lower_map = {str(c).strip().lower(): c for c in df.columns}
text_candidates = ["message", "text", "email", "email_text", "v2", "body", "content"]
label_candidates = ["label", "category", "class", "spam", "target", "v1", "type"]

text_col = next((lower_map[c] for c in text_candidates if c in lower_map), None)
label_col = next((lower_map[c] for c in label_candidates if c in lower_map), None)

if text_col is None or label_col is None or text_col == label_col:
    raise ValueError(
        "Could not identify message and label columns. Rename them to 'Message' and 'Category', "
        "or edit text_col and label_col in this cell. Available columns: " + str(df.columns.tolist())
    )

df = df[[text_col, label_col]].rename(columns={text_col: "Message", label_col: "Category"})
df["Message"] = df["Message"].fillna("").astype(str).str.strip()
df["Category"] = df["Category"].astype(str).str.strip().str.lower()

label_map = {
    "spam": "spam", "1": "spam", "junk": "spam", "yes": "spam",
    "ham": "ham", "not spam": "ham", "not_spam": "ham", "0": "ham",
    "legitimate": "ham", "normal": "ham", "no": "ham"
}
df["Category"] = df["Category"].map(lambda x: label_map.get(x, x))
df = df[df["Category"].isin(["spam", "ham"])].copy()
df = df[df["Message"].str.len() > 0].copy()
df["Spam"] = (df["Category"] == "spam").astype(int)

print("Detected message column:", "Message")
print("Detected label column:", "Category")
print("\nMissing values after cleaning:")
display(df.isnull().sum().to_frame("Missing count"))
print("Dataset shape after cleaning:", df.shape)
print("\nLabel counts:")
display(df["Category"].value_counts().to_frame("Count"))
```

```
Missing values before cleaning:
```

	Missing count
text	0
spam	0

```
Duplicate rows before cleaning: 33
Detected message column: Message
Detected label column: Category

Missing values after cleaning:
```

	Missing count
Message	0
Category	0
Spam	0

```
Dataset shape after cleaning: (5695, 3)

Label counts:
```

Category	Count
ham	4327
spam	1368

3. Descriptive statistics

What we use

- Mean: average value.
- Median: middle value after sorting.
- Mode: most frequent value.
- Variance and standard deviation: spread of values.
- Message length and word count: simple numerical features for email text.

```
df["Character_Count"] = df["Message"].str.len()
df["Word_Count"] = df["Message"].apply(lambda x: len(x.split()))
numeric_cols = ["Character_Count", "Word_Count", "Spam"]

print("Descriptive statistics:")
display(df[numeric_cols].describe().T)

for col in ["Character_Count", "Word_Count"]:
    print(f"\n{col}")
    print("Mean:", round(df[col].mean(), 2))
    print("Median:", round(df[col].median(), 2))
    print("Mode:", round(df[col].mode().iloc[0], 2))
    print("Variance:", round(df[col].var(), 2))
    print("Standard deviation:", round(df[col].std(), 2))
    print("Q1 (25%):", round(df[col].quantile(0.25), 2))
    print("Q3 (75%):", round(df[col].quantile(0.75), 2))

print("\nStatistics by category:")
display(df.groupby("Category")[["Character_Count", "Word_Count"]].describe().round(2))
```

```
Descriptive statistics:
```

	count	mean	std	min	25%	50%	75%	max
Character_Count	5695.0	1557.955048	2047.072170	12.0	508.0	979.0	1892.5	43952.0
Word_Count	5695.0	327.079543	419.598400	2.0	101.0	210.0	401.0	8477.0
Spam	5695.0	0.240211	0.427249	0.0	0.0	0.0	0.0	1.0

```
Character_Count
Mean: 1557.96
Median: 979.0
Mode: 445
Variance: 4190504.47
Standard deviation: 2047.07
Q1 (25%): 508.0
Q3 (75%): 1892.5

Word_Count
Mean: 327.08
Median: 210.0
Mode: 75
Variance: 176062.82
Standard deviation: 419.6
Q1 (25%): 101.0
Q3 (75%): 401.0

Statistics by category:
```

	Character_Count								Word_Count							
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min	25%	50%	75%	max
Category																
ham	4327.0	1634.17	1965.01	12.0	577.5	1122.0	2037.5	43952.0	4327.0	346.25	407.19	2.0	119.0	239.0	439.5	8477.0
spam	1368.0	1316.88	2271.33	17.0	401.5	693.0	1249.5	28431.0	1368.0	266.43	451.46	4.0	79.0	139.5	251.0	6129.0

4. Exploratory Data Analysis (EDA) and graphs

What we use

- Histogram: shows message-length distribution.
- Boxplot: compares message lengths and possible outliers.
- Bar chart: compares the number of spam and ham messages.
- Scatterplot: explores the relationship between word count and character count.

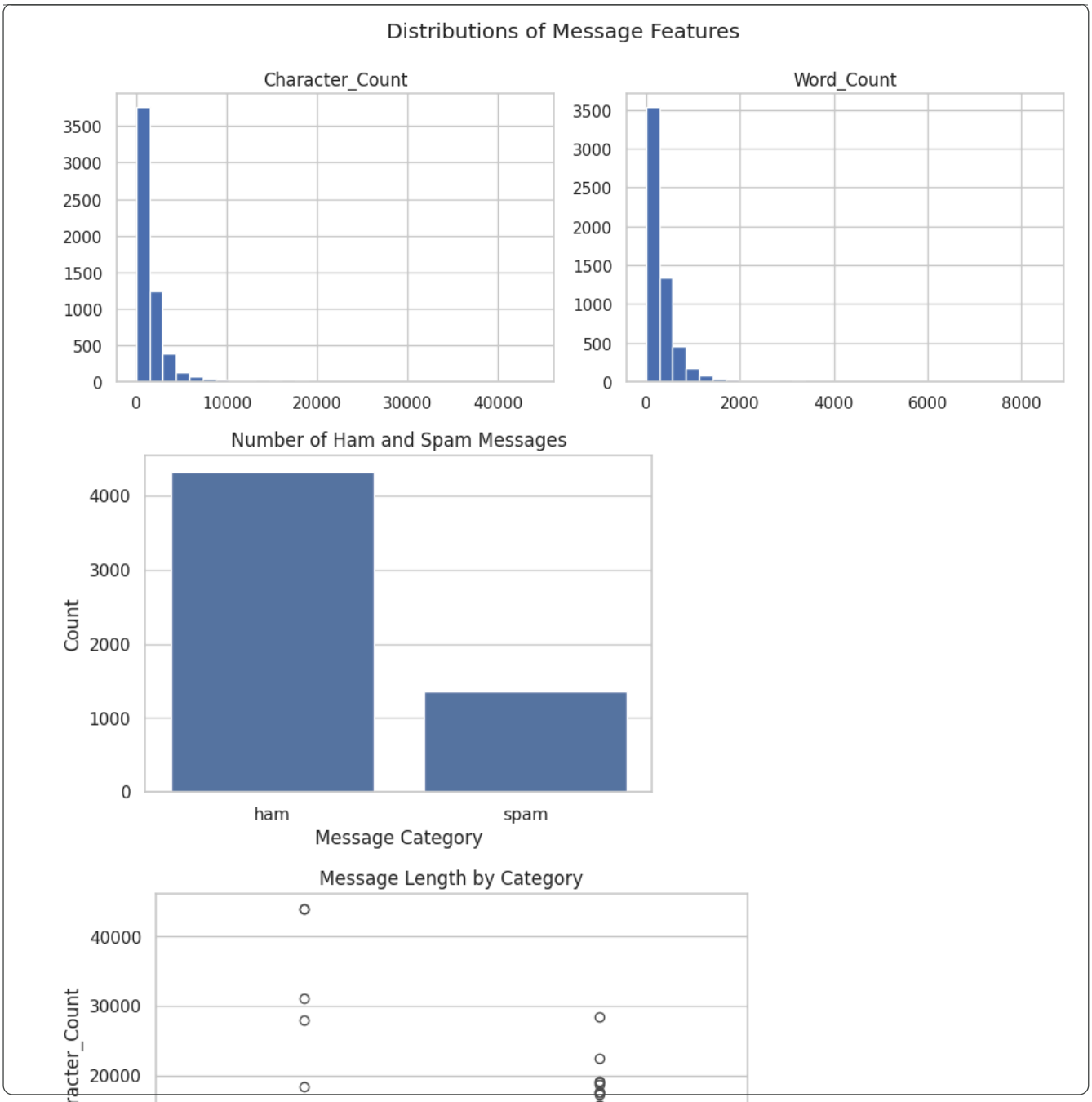
```
# Distribution of message lengths
df[["Character_Count", "Word_Count"]].hist(figsize=(10, 4), bins=30)
plt.suptitle("Distributions of Message Features")
plt.tight_layout()
plt.show()

# Spam vs ham count
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="Category", order=["ham", "spam"])
plt.title("Number of Ham and Spam Messages")
plt.xlabel("Message Category")
plt.ylabel("Count")
plt.show()

# Message length by category
plt.figure(figsize=(7, 4))
sns.boxplot(data=df, x="Category", y="Character_Count", order=["ham", "spam"])
plt.title("Message Length by Category")
plt.show()

# Scatterplot
plt.figure(figsize=(7, 5))
sns.scatterplot(data=df.sample(min(len(df), 2000), random_state=42),
                x="Word_Count", y="Character_Count", hue="Category", alpha=0.6)
plt.title("Word Count vs Character Count")
plt.show()

# Average message length by category
plt.figure(figsize=(6, 4))
sns.barplot(data=df, x="Category", y="Character_Count", order=["ham", "spam"], errorbar=("ci", 95))
plt.title("Average Character Count by Category")
plt.show()
```



5. Probability distributions, sampling, and CLT

What we use

- Histogram: checks observed message lengths.
- Uniform and binomial distributions: demonstrate probability distributions.
- Random sample: compares a sample mean with the full dataset mean.
- Central Limit Theorem (CLT): sample means tend to form an approximately normal distribution when repeated sampling conditions are suitable.

```
# Observed character-count distribution
plt.figure(figsize=(7, 4))
sns.histplot(df["Character_Count"], bins=30, kde=True)
plt.title("Observed Message Length Distribution")
plt.xlabel("Character count")
plt.show()

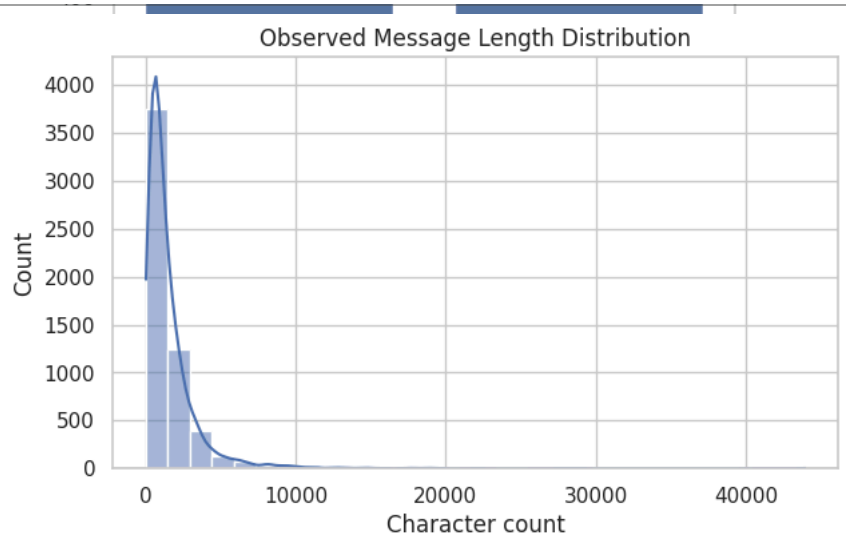
# Simulated uniform distribution
uniform_data = np.random.uniform(low=0, high=1, size=1000)
plt.figure(figsize=(7, 4))
plt.hist(uniform_data, bins=20, edgecolor="black")
plt.title("Simulated Uniform Distribution")
plt.show()

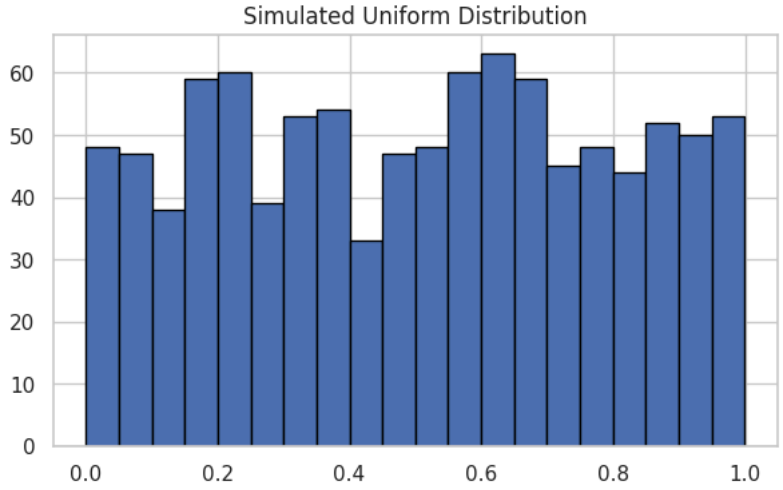
# Simulated binomial distribution
binomial_data = np.random.binomial(n=10, p=0.5, size=1000)
```

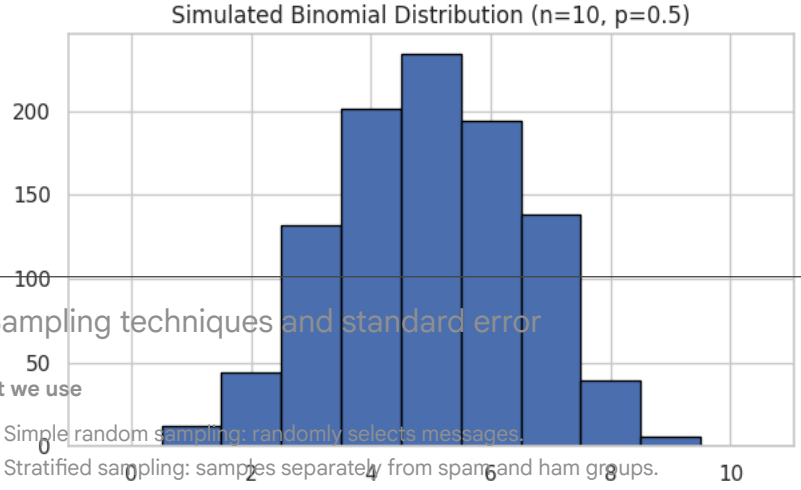
```
plt.figure(figsize=(7, 4))
plt.hist(binomial_data, bins=np.arange(-0.5, 11.5, 1), edgecolor="black")
plt.title("Simulated Binomial Distribution (n=10, p=0.5)")
plt.xlabel("Number of successes")
plt.show()

sample = df["Character_Count"].sample(n=min(100, len(df)), random_state=42)
print("Population mean character count:", round(df["Character_Count"].mean(), 2))
print("Random sample mean (n={}):".format(len(sample)), round(sample.mean(), 2))

rng = np.random.default_rng(42)
values = df["Character_Count"].to_numpy()
sample_means = [rng.choice(values, size=min(40, len(values)), replace=True).mean() for _ in range(1000)]
plt.figure(figsize=(7, 4))
plt.hist(sample_means, bins=25, edgecolor="black")
plt.title("Sampling Distribution of Mean Character Count (CLT Demonstration)")
plt.xlabel("Sample mean")
plt.show()
```







6. Sampling techniques and standard error

What we use

- Simple random sampling: randomly selects messages.
- Stratified sampling: samples separately from spam and ham groups.
- Standard error: estimates how much the sample mean varies between samples.

```
Population mean character count: 1557.96
```

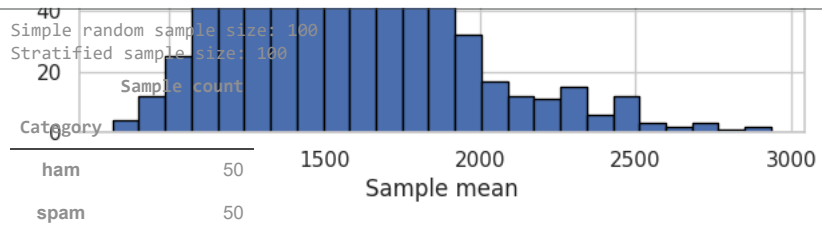
```
simple_sample = df.sample(n=min(100, len(df)), random_state=42)
print("Simple random sample size:", len(simple_sample))

stratified_sample = (
    df.groupby("Category", group_keys=False)
      .apply(lambda group: group.sample(n=min(50, len(group)), random_state=42))
)
print("Stratified sample size:", len(stratified_sample))
display(stratified_sample.groupby("Category").size().to_frame("Sample count"))

se = stats.sem(df["Character_Count"]) if len(df) > 1 else 0
print("Standard error of mean character count:", round(se, 3))
```

```
Simple random sample size: 100
Stratified sample size: 100
```

Category	Sample count
ham	50
spam	50




```
Standard error of mean character count: 27.126
```

7. Confidence interval for the mean

A 95% confidence interval estimates a range for the population mean message length based on this sample.

```
confidence = 0.95
mean_length = df["Character_Count"].mean()
standard_error = stats.sem(df["Character_Count"]) if len(df) > 1 else 0

if len(df) > 1:
    ci_low, ci_high = stats.t.interval(
        confidence, df=len(df) - 1, loc=mean_length, scale=standard_error
    )
    print(f"Mean message length: {mean_length:.2f} characters")
    print(f"95% confidence interval: ({ci_low:.2f}, {ci_high:.2f}) characters")
else:
    print("At least two messages are required to calculate a confidence interval.")
```

```
Mean message length: 1557.96 characters
95% confidence interval: (1504.78, 1611.13) characters
```

8. Hypothesis tests (significance level α = 0.05)

What we test

- One-sample t-test: whether average message length differs from a chosen reference value of 100 characters.
- Independent t-test (Welch): whether average message length differs between spam and ham.
- Chi-square test: whether the message category is associated with a long-message flag (at least 100 characters).

The 100-character threshold is a project-defined example, not a universal spam rule.

```
alpha = 0.05

# A. One-sample t-test against an example reference length of 100 characters
reference_chars = 100
t_stat, p_value = stats.ttest_1samp(df["Character_Count"], popmean=reference_chars)
print("ONE-SAMPLE T-TEST")
print("H0: Mean message length = 100 characters")
print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")

# B. Independent t-test: compare spam vs ham message length
spam_lengths = df.loc[df["Category"] == "spam", "Character_Count"]
ham_lengths = df.loc[df["Category"] == "ham", "Character_Count"]
if len(spam_lengths) > 1 and len(ham_lengths) > 1:
    t_stat, p_value = stats.ttest_ind(spam_lengths, ham_lengths, equal_var=False)
    print("\nWELCH TWO-SAMPLE T-TEST (Spam vs Ham)")
    print(f"Spam mean = {spam_lengths.mean():.2f}; Ham mean = {ham_lengths.mean():.2f}")
    print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.5f}")
    print("Decision:", "Reject H0" if p_value < alpha else "Fail to reject H0")
else:
    print("Not enough spam/ham observations for the independent t-test.")
```

```
# C. Chi-square test: category vs long-message flag
from scipy.stats import chi2_contingency
df["Long_Message"] = np.where(df["Character_Count"] >= 100, "100+ chars", "Under 100 chars")
contingency = pd.crosstab(df["Category"], df["Long_Message"])
display(contingency)
if contingency.shape == (2, 2):
    chi2, p_value, dof, expected = chi2_contingency(contingency)
    print("\nCHI-SQUARE TEST")
    print(f"chi-square = {chi2:.3f}, p-value = {p_value:.5f}, degrees of freedom = {dof}")
    print("Decision:", "Reject H0: category and length group are associated" if p_value < alpha
          else "Fail to reject H0: insufficient evidence of association")
```

```
ONE-SAMPLE T-TEST
H0: Mean message length = 100 characters
t-statistic = 53.747, p-value = 0.00000
Decision: Reject H0

WELCH TWO-SAMPLE T-TEST (Spam vs Ham)
Spam mean = 1316.88; Ham mean = 1634.17
t-statistic = -4.646, p-value = 0.00000
Decision: Reject H0
```

Long_Message	100+ chars	Under 100 chars
Category		
ham	4274	53
spam	1346	22

```
CHI-SQUARE TEST
chi-square = 0.899, p-value = 0.34313, degrees of freedom = 1
Decision: Fail to reject H0: insufficient evidence of association
```

9. Correlation and significance tests

Correlation measures the relationship between numerical features. It does not prove causation.

```
pairs = [("Character_Count", "Word_Count"), ("Character_Count", "Spam"), ("Word_Count", "Spam")]

print("Correlation matrix:")
display(df[["Character_Count", "Word_Count", "Spam"]].corr(method="pearson"))
print("\nCovariance matrix:")
display(df[["Character_Count", "Word_Count", "Spam"]].cov())

for x_col, y_col in pairs:
    pearson_r, pearson_p = stats.pearsonr(df[x_col], df[y_col])
    spearman_r, spearman_p = stats.spearmanr(df[x_col], df[y_col])
    print(f"\n{x_col} vs {y_col}")
    print(f"Pearson r = {pearson_r:.3f}, p-value = {pearson_p:.5f}")
    print(f"Spearman rho = {spearman_r:.3f}, p-value = {spearman_p:.5f}")
```

```
Correlation matrix:
```

	Character_Count	Word_Count	Spam
Character_Count	1.000000	0.986021	-0.066223
Word_Count	0.986021	1.000000	-0.081276
Spam	-0.066223	-0.081276	1.000000

```
Covariance matrix:
```

	Character_Count	Word_Count	Spam
Character_Count	4.190504e+06	846940.916643	-57.919302
Word_Count	8.469409e+05	176062.817170	-14.570568
Spam	-5.791930e+01	-14.570568	0.182542

```
Character_Count vs Word_Count
Pearson r = 0.986, p-value = 0.00000
Spearman rho = 0.992, p-value = 0.00000

Character_Count vs Spam
Pearson r = -0.066, p-value = 0.00000
Spearman rho = -0.188, p-value = 0.00000

Word_Count vs Spam
Pearson r = -0.081, p-value = 0.00000
Spearman rho = -0.193, p-value = 0.00000
```

10. Prepare data for machine learning

What we use

- `train_test_split`: separates training and testing data.
- `TfidfVectorizer`: converts message text into numerical features.
- `Pipeline`: combines text processing and model training.
- Stratified split: keeps spam/ham proportions similar in train and test sets.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, ConfusionMatrixDisplay
)
import joblib

X = df["Message"].astype(str)
y = df["Category"].astype(str)

if y.nunique() < 2:
    raise ValueError("The dataset must contain both spam and ham examples.")
if y.value_counts().min() < 2:
    raise ValueError("Each category needs at least 2 examples for a stratified train/test split.")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training messages:", len(X_train))
print("Testing messages:", len(X_test))
print("\nTraining label counts:")
display(y_train.value_counts().to_frame("Count"))
```

```
Training messages: 4556
Testing messages: 1139

Training label counts:
```

Category	Count
ham	3462
spam	1094

11. Train and compare classification models

Models are compared using accuracy, precision, recall, and F1-score. For spam detection, precision and recall are important in addition to accuracy.

```
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Linear SVM": LinearSVC(class_weight="balanced", random_state=42)
}

results = []
trained_models = {}

for name, classifier in models.items():
    pipeline = Pipeline(steps=[
        ("tfidf", TfidfVectorizer(lowercase=True, stop_words="english",
                                  max_features=20000, ngram_range=(1, 2))),
        ("model", classifier)
    ])
    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)

    results.append({
        "Model": name,
```

```
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision (spam)": precision_score(y_test, predictions, pos_label="spam", zero_division=0),
        "Recall (spam)": recall_score(y_test, predictions, pos_label="spam", zero_division=0),
        "F1 (spam)": f1_score(y_test, predictions, pos_label="spam", zero_division=0)
    })
    trained_models[name] = pipeline

results_df = pd.DataFrame(results).sort_values("F1 (spam)", ascending=False)
display(results_df.round(4))

best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]
print("Best model by spam F1-score:", best_model_name)
```

	Model	Accuracy	Precision (spam)	Recall (spam)	F1 (spam)
2	Linear SVM	0.9947	0.9891	0.9891	0.9891
1	Logistic Regression	0.9895	0.9645	0.9927	0.9784
0	Multinomial Naive Bayes	0.9789	1.0000	0.9124	0.9542

```
Best model by spam F1-score: Linear SVM
```

12. Confusion matrix and classification report

- **True Positive (TP):** spam correctly identified as spam.
- **True Negative (TN):** ham correctly identified as ham.
- **False Positive (FP):** ham incorrectly marked as spam.
- **False Negative (FN):** spam incorrectly marked as ham.

```
best_predictions = best_model.predict(X_test)

print("CLASSIFICATION REPORT")
print(classification_report(y_test, best_predictions, labels=["ham", "spam"], zero_division=0))

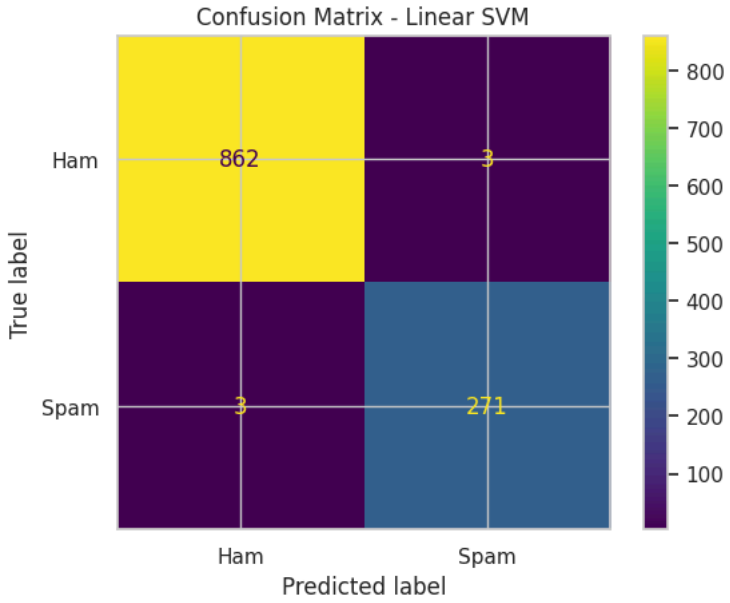
cm = confusion_matrix(y_test, best_predictions, labels=["ham", "spam"])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Ham", "Spam"])
disp.plot(values_format="d")
plt.title(f"Confusion Matrix - {best_model_name}")
plt.show()

comparison = pd.DataFrame({
    "Actual category": y_test.to_numpy(),
    "Predicted category": best_predictions
})
display(comparison.head(10))
```

```
CLASSIFICATION REPORT
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00       865
        spam       0.99      0.99      0.99       274

    accuracy                           0.99      1139
   macro avg       0.99      0.99      0.99      1139
weighted avg       0.99      0.99      0.99      1139
```



	Actual category	Predicted category
0	ham	ham
1	ham	ham
2	ham	ham
3	ham	ham
4	ham	ham
5	ham	ham
6	ham	ham
7	ham	ham
8	spam	spam
9	spam	spam

13. Save the best model

The trained pipeline includes TF-IDF text processing and the classifier, so it can be loaded to predict new messages.

```
# Save the selected best model in both Pickle (.pkl) and Joblib (.joblib) formats
# The full Pipeline includes TF-IDF text processing and the trained classifier.

import pickle
import joblib

pkl_filename = "best_spam_email_model.pkl"
joblib_filename = "best_spam_email_model.joblib"

# Save as PKL using Python's pickle module
with open(pkl_filename, "wb") as file:
    pickle.dump(best_model, file)

# Also save as Joblib (optional alternative)
joblib.dump(best_model, joblib_filename)

print("Saved PKL model:", pkl_filename)
print("Saved Joblib model:", joblib_filename)

# Verify that the PKL file can be loaded
with open(pkl_filename, "rb") as file:
    pkl_loaded_model = pickle.load(file)
```

```
# Quick prediction test using one test message
sample_message = [str(X_test.iloc[0])]
print("Sample prediction from reloaded PKL:", pkl_loaded_model.predict(sample_message)[0])

# Download the PKL file to your computer
from google.colab import files
files.download(pkl_filename)
```

```
Saved PKL model: best_spam_email_model.pkl
Saved Joblib model: best_spam_email_model.joblib
Sample prediction from reloaded PKL: ham
```

14. Interactive prediction interface using Gradio

Enter or paste a message to classify it as spam or ham. The interface uses the model trained in this notebook.

```
!pip -q install gradio
```

```
import gradio as gr
import pickle

# Load the trained pipeline from the PKL file
with open("best_spam_email_model.pkl", "rb") as file:
    loaded_model = pickle.load(file)

def predict_spam(message):
    message = str(message).strip()
    if not message:
        return "Please enter an email or message."
    prediction = loaded_model.predict([message])[0]
    if prediction == "spam":
        return "Prediction: SPAM 🚨"
    return "Prediction: HAM (Not Spam) ✅"

demo = gr.Interface(
    fn=predict_spam,
    inputs=gr.Textbox(lines=6, label="Email / Message", placeholder="Paste the email text here..."),
    outputs=gr.Textbox(label="Detection Result"),
    title="Spam Email Detection",
    description="Enter an email or message to check whether it is spam or not spam."
)
demo.launch(share=True)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
```